APUNTES UNIVERSITARIOS

Foundations of Machine Learning and Deep Learning

Apuntes del Tema 3 y Tema 4: Modelos Lineales — Regresión Lineal y Regresión Logística

18 de octubre de 2026

Índice general

1 Apuntes del Tema 3 y Tema 4: Modelos Lineales — Regresión Lineal y Regresión Logística **2**

1.1 1. Regresión Lineal 2

1.1.1 1.1 Introducción y Planteamiento del Modelo 2

1.1.2 1.2 Función de Pérdida (Error Cuadrático Medio - MSE) 2

1.1.3 1.3 Visión Probabilística: Derivación desde Máxima Verosimilitud (MLE) 2

1.1.4 1.4 Solución Analítica vs. Descenso de Gradiente 2

1.2 2. Regresión Logística 3

1.2.1 2.1 Concepto y Función Sigmoide 3

1.2.2 2.2 Función de Pérdida (Entropía Cruzada Binaria / *Binary Cross-Entropy*) 3

1.2.3 2.3 Visión Probabilística: Derivación desde la Distribución de Bernoulli 3

1.3 3. Escalado de Características y Expansión Polinomial 4

1.3.1 3.1 Importancia del Escalado de Características (*Feature Scaling*) 4

1.3.2 3.2 Expansión Polinomial (*Polynomial Features*) 4

1.4 4. Regularización en Modelos Lineales (L_1 y L_2) 4

1.5 5. Práctica y Aplicación en Código (PyTorch y Scikit-Learn) 5

1 Apuntes del Tema 3 y Tema 4: Modelos Lineales — Regresión Lineal y Regresión Logística

1.1 1. Regresión Lineal

1.1.1 1.1 Introducción y Planteamiento del Modelo

- **Definición:** Modelo paramétrico supervisado para predecir un valor objetivo continuo $y \in \mathbb{R}$ a partir de un vector de características $x \in \mathbb{R}^d$.
- **Ecuación del Modelo:**

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b = x^\top w + b$$

donde $w \in \mathbb{R}^d$ es el vector de pesos (pesos de las características) y $b \in \mathbb{R}$ es el sesgo (*bias* o intersección).

- **Interpretación Geométrica:** Representa el ajuste de un hiperplano d-dimensional embebido en un espacio de dimensión $\mathbb{R}^{d+1}$:
- $d = 1$: Una recta de regresión.
- $d = 2$: Un plano de regresión.

1.1.2 1.2 Función de Pérdida (Error Cuadrático Medio - MSE)

Para medir la discrepancia entre las predicciones $\hat{y}^{(i)}$ y los valores reales $y^{(i)}$, se define el residuo $r^{(i)} = y^{(i)} - \hat{y}^{(i)}$.

La función de pérdida promedio sobre un dataset de n muestras es el **MSE (Mean Squared Error)**:

$$\mathcal{L}(w, b) = \frac{1}{n} \sum_{i=1}^{n} \left(y^{(i)} - \hat{y}^{(i)} \right)^2 = \frac{1}{n} \| y - (Xw + b\mathbf{1}) \|_2^2$$

1.1.3 1.3 Visión Probabilística: Derivación desde Máxima Verosimilitud (MLE)

El uso del error cuadrático medio no es arbitrario; surge de asumir que el valor real y se genera mediante una relación lineal determinista más un **ruido gaussiano** centrado en cero $\epsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$:

$$y^{(i)} = (x^{(i)})^\top w + b + \epsilon^{(i)} \implies y^{(i)} \mid x^{(i)} \sim \mathcal{N}\left((x^{(i)})^\top w + b,\, \sigma^2 \right)$$

Al maximizar la función de Log-Verosimilitud (*Log-Likelihood*) de este modelo bajo muestras independientes e idénticamente distribuidas (*i.i.d.*), la maximización equivale exactamente a la **minimización del MSE**.

1.1.4 1.4 Solución Analítica vs. Descenso de Gradiente

- **Solución Analítica (Ecuación Normal):**

Añadiendo una columna de unos a la matriz de características ($\tilde{X} = [\mathbf{1}, X]$), se obtiene el mínimo global en un solo paso:

$$\tilde{w} = (\tilde{X}^\top \tilde{X})^{-1} \tilde{X}^\top y$$

- *Inconveniente:* Invertir la matriz $\tilde{X}^\top \tilde{X}$ requiere un coste computacional $O(d^3)$, siendo inviable para dimensiones elevadas (d muy grande).
- **Descenso de Gradiente (Iterativo):**

Actualización iterativa mediante las derivadas parciales de la pérdida:

$$\frac{\partial \mathcal{L}}{\partial w_j} = -\frac{2}{n} \sum_{i=1}^{n} \left(y^{(i)} - \hat{y}^{(i)} \right) x_j^{(i)}, \quad \frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{n} \sum_{i=1}^{n} \left(y^{(i)} - \hat{y}^{(i)} \right)$$

$$w_j \leftarrow w_j - \eta \frac{\partial \mathcal{L}}{\partial w_j}, \quad b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b}$$

1.2 2. Regresión Logística

1.2.1 2.1 Concepto y Función Sigmoide

- **Definición:** Modelo paramétrico supervisado para **clasificación binaria** que predice la probabilidad $p = P(y = 1 \mid x) \in (0, 1)$.
- **Transformación Sigmoide / Logística:**

Para mapear el valor continuo lineal $z = x^\top w + b$ (denominado *logit*) al rango probabilístico $(0, 1)$, se aplica la función sigmoide $\sigma(z)$:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- **Frontera de Decisión (*Decision Boundary*):**

El umbral estándar $P(y = 1 \mid x) = 0.5$ corresponde a $z = 0$ ($x^\top w + b = 0$), formando un hiperplano lineal de separación en el espacio de características.

1.2.2 2.2 Función de Pérdida (Entropía Cruzada Binaria / *Binary Cross-Entropy*)

Utilizar MSE sobre salidas sigmoideas genera una superficie de pérdida no convexa llena de zonas llanas (*plateaus*) donde el gradiente se desvanece ($\nabla \mathcal{L} \approx 0$). Por ello, se emplea la **Pérdida de Entropía Cruzada Binaria**:

$$\mathcal{L}(y, \hat{y}) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}) \right]$$

- **Propiedades:** Es strictly convexa respecto a los parámetros (w, b), garantizando que el descenso de gradiente alcance el mínimo global sin quedar atrapado en mínimos locales subóptimos.

1.2.3 2.3 Visión Probabilística: Derivación desde la Distribución de Bernoulli

Al asumir que la etiqueta objetivo $y \in \{0, 1\}$ sigue una **distribución de Bernoulli** paramétrica con probabilidad de éxito $p = \hat{y} = \sigma(z)$:

$$P(y \mid x) = \hat{y}^y (1 - \hat{y})^{(1-y)}$$

La función de Entropía Cruzada equivale numéricamente al negativo de la función de Log-Verosimilitud (*Negative Log-Likelihood - NLL*). Por consiguiente, ajustar la Regresión Logística mediante Entropía Cruzada es equivalente a realizar la **Estimación de Máxima Verosimilitud (MLE)** de un proceso de Bernoulli.

1.3 3. Escalado de Características y Expansión Polinomial

1.3.1 3.1 Importancia del Escalado de Características (*Feature Scaling*)

Cuando las características poseen órdenes de magnitud desiguales, las curvas de nivel de la función de pérdida se deforman formando elipses alargadas. Esto provoca que el descenso de gradiente oscile en un patrón ineficiente en *zig-zag*, retrasando drásticamente la convergencia.

- **Estandarización (Standardization):**

$$x_j \mapsto \frac{x_j - \mu_j}{\sigma_j}$$

Transforma cada característica para que tenga media $\mu = 0$ y varianza unitaria $\sigma^2 = 1$, volviendo las curvas de nivel más circulares y estabilizando las actualizaciones de los gradientes.

1.3.2 3.2 Expansión Polinomial (*Polynomial Features*)

Cuando las clases no son separables por una frontera lineal recta, es posible mapear el espacio de características original a un espacio de mayor dimensión introduciendo combinaciones polinomiales.

- **Ejemplo para 2 características (x_1, x_2) a grado $d = 2$:**

$$z = w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_2^2 + w_5 x_1 x_2 + b$$

- **Ventajas e Inconvenientes:**
- Permite capturar límites de decisión no lineales complejas (elipses, parábolas) conservando la linealidad en los parámetros.
- Crecimiento exponencial del número de características al aumentar el grado d, elevando drásticamente el riesgo de sobreajuste (*overfitting*) y haciendo indispensable el escalado.

1.4 4. Regularización en Modelos Lineales (L_1 y L_2)

Para evitar que el modelo asigne pesos desproporcionadamente grandes a las características (previniendo el sobreajuste), se añade un término de penalización sobre los pesos a la función de pérdida:

$$\mathcal{J}(w, b) = \mathcal{L}(w, b) + \Omega(w)$$

Regularización	Término de Penalización $\Omega(w)$	Forma Geométrica	Comportamiento del Gradiente y Solución
L_2 **(Ridge)**	$\frac{\lambda}{2}\|w\|_2^2 = \frac{\lambda}{2}\sum_{j=1}^{d} w_j^2$	Esfera / Círculo suave	Reduce la magnitud de todos los pesos proporcionalmente ($w \leftarrow w(1-\eta\lambda)$), pero **rara vez los hace exactamente cero**.
L_1 **(Lasso)**	$\lambda\|w\|_1 = \lambda\sum_{j=1}^{d} \|w_j\|$	Diamante / Hiperoctaedro con esquinas en los ejes	Mantiene una magnitud de gradiente constante ($\lambda \cdot \text{signo}(w)$), **conduciendo pesos exactamente a cero** y generando soluciones esparsas (*feature selection*).

1.5 5. Práctica y Aplicación en Código (PyTorch y Scikit-Learn)

```
1
2  import torch
3
4  import torch.nn as nn
5
6  import torch.optim as optim
7
8  from sklearn.linear_model import LogisticRegression
9
10 from sklearn.preprocessing import StandardScaler, PolynomialFeatures
11
12 from sklearn.pipeline import Pipeline
13
14
15
16 # ---------------------------------------------------------------------
17
18 # 1. Definición del Modelo de Regresión Logística en PyTorch
19
20 # ---------------------------------------------------------------------
21
22 class LogisticRegressionPyTorch(nn.Module):
23
24     def __init__(self, input_dim):
25
26         super(LogisticRegressionPyTorch, self).__init__()
27
28         # Capa lineal que calcula el logit z = xW + b
29
30         self.linear = nn.Linear(input_dim, 1)
31
32
```

```
33
34      def forward(self, x):
35
36          # Retorna el logit directamente.
37
38          # BCEWithLogitsLoss aplica la sigmoide de forma numéricamente estable.
39
40          return self.linear(x)
41
42
43
44  # ----------------------------------------------------------------------------
45
46  # 2. Configuración y Bucle de Entrenamiento
47
48  # ----------------------------------------------------------------------------
49
50  def train_logistic_regression(X_train_tensor, y_train_tensor, num_epochs=100, lr=0.01,
        weight_decay=1e-4):
51
52      device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
53
54
55
56      # Instanciación
57
58      input_dim = X_train_tensor.shape[1]
59
60      model = LogisticRegressionPyTorch(input_dim).to(device)
61
62
63
64      # Función de pérdida (Binary Cross Entropy con Sigmoide integrada)
65
66      criterion = nn.BCEWithLogitsLoss()
67
68
69
70      # Optimizador SGD con L2 Regularization (weight_decay)
71
72      optimizer = optim.SGD(model.parameters(), lr=lr, weight_decay=weight_decay)
73
74
75
76      X_train_tensor = X_train_tensor.to(device)
77
78      y_train_tensor = y_train_tensor.to(device).unsqueeze(1) # (B, 1)
79
80
81
82      for epoch in range(1, num_epochs + 1):
83
84          model.train()
85
86
87
88          # 1. Limpiar gradientes
89
```

```
        optimizer.zero_grad()


        # 2. Forward pass (logits)
        logits = model(X_train_tensor)
        loss = criterion(logits, y_train_tensor)


        # 3. Backward pass y actualización de parámetros
        loss.backward()
        optimizer.step()


        if epoch % 20 == 0 or epoch == 1:
            print(f"Época [{epoch:3d}/{num_epochs}] | Loss: {loss.item():.4f}")


    return model


# ------------------------------------------------------------------------
# 3. Implementación en Scikit-Learn
# ------------------------------------------------------------------------
def build_sklearn_logistic_pipeline(degree=2, penalty='l2', C=1.0):
    """
    Crea un pipeline completo con escalado, expansión polinomial y regresión logística.
    Note: C en scikit-learn es la inversa de lambda (C = 1 / lambda).
    """
    pipeline = Pipeline([
        ('scaler', StandardScaler()),
        ('poly', PolynomialFeatures(degree=degree, include_bias=False)),
        ('logistic', LogisticRegression(penalty=penalty, C=C, solver='saga', random_state=42))
    ])

```

```
146     return pipeline
```